Validación de jugadores virtuales autónomos en RoboCup Soccer Simulation 2D: avance del proyecto final

[Nombre del estudiante 1], [Nombre del estudiante 2]
[Programa académico] – [Universidad]
[Ciudad], Colombia
{correo1, correo2}@[dominio]

***Resumen*—Este avance documenta la puesta en marcha del servidor oficial RoboCup Soccer Simulation 2D (`rcssserver` 19.0.0) y la implementación de *AutonomoFC*, un equipo de once agentes escrito desde cero en Python que se comunica con el servidor por UDP, se auto-localiza triangulando banderas del campo y decide con una formación 4-3-3 dinámica, selección de interceptor por tiempo de llegada y un árbol de decisión tiro–pase–conducción–despeje. Se definen métricas de equipo, de agente y de calidad interna (error de localización, tiempo de cómputo) y se reportan resultados preliminares de 11 partidos completos: AutonomoFC vence a una línea base reactiva con un promedio de 18.6 goles a favor y 0.0 en contra, pero pierde frente a HELIOS-base (0–30, 0–35 y 0–23), lo que delimita el trabajo restante.**

***Index Terms*—RoboCup, simulación 2D, sistemas multiagente, agentes autónomos, auto-localización.**

I. Introducción

La liga de simulación 2D de RoboCup modela un partido de fútbol entre dos equipos de once agentes autónomos que solo perciben el entorno de forma parcial y ruidosa, y actúan mediante comandos discretos cada ciclo de 100 ms [1]. El objetivo de esta entrega es validar el comportamiento de jugadores virtuales propios en ese entorno: (i) evidenciar el servidor en funcionamiento, (ii) implementar un agente con estrategias autónomas distinto de los clientes de ejemplo, (iii) definir métricas de evaluación, y (iv) registrar los problemas encontrados y los siguientes pasos.

II. Entorno de simulación

El servidor se compiló desde el repositorio oficial [2] (`rcssserver-19.0.0`, commit `ce87001`) con CMake, Boost y flex/bison en Ubuntu 24.04; el visor `rcssmonitor` 19.0.1 se compiló con Qt5 [3]. El Listado 1 muestra la salida real del servidor al cerrar un partido de evaluación y la Fig. 1, una captura del monitor reproduciendo un partido de AutonomoFC.

Para las evaluaciones se usa el modo sincrónico (`server::synch_mode=true`): el servidor avanza en cuanto todos los clientes envían `(done)`, lo que permite jugar un partido completo de 2×3000 ciclos en 1–3 min sin alterar la física. El video de demostración se obtiene reproduciendo los logs del servidor en el monitor oficial. Se usan los parámetros por defecto, sin entrenador; el servidor asigna tipos de jugador heterogéneos con la semilla `player::random_seed`, distinta en cada partido.

Listado 1. Salida del servidor (extracto, partido E1, semilla 1).

```
rcssserver-19.0.0
Using given Hetero Player Seed: 1
Waiting for players to connect
A new (v19) player (AutonomoFC 1) connected.
   ... (AutonomoFC 2..11)
A new (v19) player (BaselineChaser 1) connected.
   ... (BaselineChaser 2..11)
Waiting to kick off
rcssserver changes (player AutonomoFC 2) to type 1
   ... (tipos heterogeneos 1..10)
Kick_off_left
Kick_off_right
Game Over. Exiting...
Game Results:
  2026-11-02 09:47:30
  'AutonomoFC' vs 'BaselineChaser'
  Score: 22 - 0
```

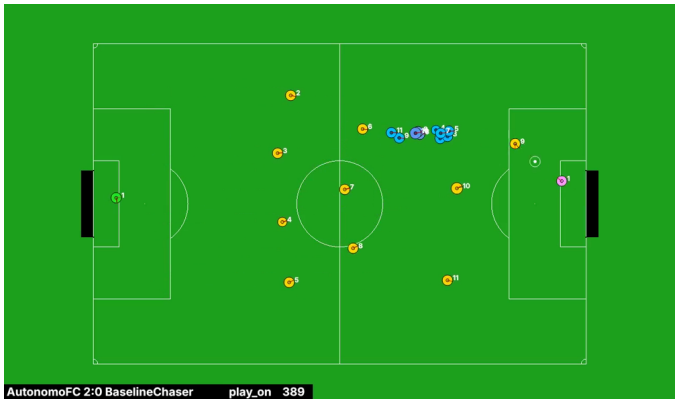


Figura 1. `rcssmonitor` reproduciendo el log `.rcg` del partido E1 (ciclo 389): AutonomoFC (amarillo) conserva su formación 4-3-3 mientras BaselineChaser (azul) se amontona sobre el balón.

III. Agente propuesto: AutonomoFC

Cada jugador es un proceso independiente (Fig. 2) que se conecta al puerto 6000 con el protocolo versión 19 y repite, cada vez que recibe `sense_body`, el ciclo percepción–modelo–decisión–acción. No se usa información global del servidor ni comunicación entre agentes: toda la coordinación surge de que cada agente aplica las mismas reglas sobre su propia percepción.

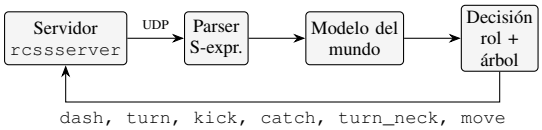


Figura 2. Arquitectura de cada agente (un proceso por jugador).

III-A. Percepción y auto-localización

El mensaje `see` entrega, para cada bandera visible, distancia d y dirección α relativas a la cabeza. Para cada par de banderas A, B con vectores relativos $\mathbf{r}_A, \mathbf{r}_B$ se estima el ángulo global de la cabeza y luego la posición:

$$\theta = \operatorname{atan2}(B - A) - \operatorname{atan2}(\mathbf{r}_B - \mathbf{r}_A), \tag{1}$$

$$\mathbf{p} = \textstyle\sum_i w_i \, (F_i - R_\theta \, \mathbf{r}_i), \tag{2}$$

promediando θ de forma circular sobre los pares bien condicionados y ponderando cada bandera con $w_i \propto 1/(1 + d_i)$, porque el ruido de cuantización crece con la distancia. Si el equipo juega por la derecha se invierte la tabla de 55 banderas, de modo que el agente siempre ataca hacia $+x$. Entre observaciones (`see` llega cada 2 ciclos en vista normal sincronizada) se integra por odometría con la velocidad de `sense_body` y el giro comandado. El balón se propaga con el decaimiento del servidor ($v_{t+1} = 0.94\, v_t$) y su velocidad se estima con `distchg`/`dirchg`.

III-B. Estrategias de decisión

Formación dinámica. Cada dorsal tiene una posición base 4-3-3 que se desplaza con el balón: $x = x_b + 0.55\, x_{bal}$, $y = 0.9\, y_b + 0.25\, y_{bal}$. Los defensas no superan al balón en más de 5 m y los delanteros se mantienen 1 m detrás de la línea de fuera de juego estimada (penúltimo rival visto).

Selección de interceptor. Cada agente predice la trayectoria del balón y calcula el primer ciclo n en que puede alcanzarlo; solo persigue el balón si ningún compañero visible llega al menos 1.5 ciclos antes. Esto evita el "enjambre" típico de los equipos reactivos.

Árbol con balón. (1) *Tiro* si la distancia al arco es < 22 m, al poste opuesto al portero rival; (2) *pase* si hay presión (rival a < 5 m), un balón parado propio, o un compañero claramente mejor ubicado: se evalúa cada compañero entre 5 y 32 m con un criterio de avance, espacio libre del receptor y un cono de intercepción (rival a menos de $1.5 + 0.35\, s$ m de la línea a la distancia s); (3) *despeje* en el tercio defensivo bajo presión; (4) *conducción* con toques de 0.85 m/ciclo en la dirección (entre $\pm 60°$) que maximiza avance y distancia al rival más cercano. La potencia de pateo se calcula invirtiendo el modelo del servidor (`kick_power_rate` con penalización por ángulo y distancia al balón).

Sin balón. Marcaje: en campo propio, defensas y medios cubren al rival más peligroso cerca de su zona, interponiéndose hacia el arco. **Portero:** se ubica sobre la bisectriz balón–arco a 1.5–5 m de la línea, ataja (`catch`) dentro del área y saca con pase seguro o hacia la banda. **Estamina:** la potencia de carrera se limita a 70 y 40 por debajo de 4500 y 3000 unidades, y los no interceptores trotan si están a menos de 8 m de su posición. **Visión:** el cuello sigue el balón o barre $\pm 90°$ cuando se pierde. Tras sacar un balón parado el agente no vuelve a tocarlo (falta por doble toque).

Tabla I
MÉTRICAS PARA EVALUAR A CADA AGENTE Y AL EQUIPO

Nivel	Métrica	Qué valida
Equipo	Goles a favor/en contra	Eficacia global
	Posesión (%)	Control del balón (último en tocarlo)
	Balón en campo rival (%)	Dominio territorial
	Tiros y tiros al arco	Generación de ocasiones
Agente	Precisión de pase (%)	Calidad de la evaluación de pases
	Distancia recorrida (m)	Movilidad / posicionamiento
	Estamina mínima media	Gestión de energía
	Fueras de juego, faltas	Respeto de reglas
Interna	Error localización P50/P90	Calidad del modelo del mundo
	Error orientación P50	Calidad del modelo del mundo
	Tiempo de decisión (ms)	Viabilidad en tiempo real

III-C. Equipos de comparación

BaselineChaser es una línea base propia con la misma percepción pero decisión reactiva: todos persiguen el balón y patean al arco a máxima potencia, lo que aísla el efecto de la capa de decisión. *HELIOS-base* [4], compilado con `librcsc`, es el equipo de referencia abierto de la comunidad y sirve como cota superior.

IV. MÉTRICAS DE EVALUACIÓN

La Tabla I resume las métricas, todas calculadas automáticamente desde el log del servidor (`.rcg`, verdad de terreno) y los registros propios de cada agente (`analisis/metrics.py`). Un pase se cuenta como completado cuando el siguiente toque de balón (detectado por el contador de `kick` del servidor con el jugador a menos de 1.25 m del balón) es de un compañero; un tiro, cuando el balón sale a más de 1.6 m/ciclo hacia la boca del arco desde $x > 30$.

V. RESULTADOS PRELIMINARES

Se jugaron 11 partidos completos: E1, AutonomoFC por la izquierda contra BaselineChaser (5 semillas); E2, lados invertidos (3 semillas), para comprobar que la inversión de coordenadas funciona; y E3, contra HELIOS-base (3 semillas). La Tabla II y la Fig. 3 muestran media $\pm$ desviación.

Frente a la línea base (E1, E2) AutonomoFC gana los 8 partidos, con 76 % de posesión y el balón en campo rival el 81 % del tiempo de juego. Las métricas casi no cambian al invertir los lados, lo que valida la transformación de coordenadas. La gestión de estamina se refleja en la estamina mínima media (unas 3900 unidades frente a 540 de BaselineChaser, que agota a sus jugadores persiguiendo el balón). La precisión de pase es solo moderada (52 %) porque el criterio cuenta como pase todo toque seguido de otro jugador, incluidos despejes y conducciones largas.

Frente a HELIOS-base (E3) el equipo pierde los tres partidos (media 0–29.3), no logra ningún tiro y su precisión de

Tabla II
RESULTADOS PRELIMINARES: AUTONOMOFC (A) FRENTE AL RIVAL (R), MEDIA ± DESVIACIÓN

	E1: BaselineChaser (n=5)		E2: BaselineChaser (n=3)		E3: HELIOS-base (n=3)	
Métrica	A	R	A	R	A	R
Marcadores	22-0, 18-0, 17-0, 15-0, 21-0		19-0, 16-2, 16-0		0-30, 0-35, 0-23	
Goles	18.6±2.6	0.0±0.0	17.0±1.4	0.7±0.9	0.0±0.0	29.3±4.9
Posesión (%)	76±2	24±2	76±1	24±1	33±5	67±5
Balón en campo rival (%)	81±3	19±3	81±5	19±5	12±3	88±3
Tiros	23.6±4.4	0.2±0.4	21.0±0.8	0.3±0.5	0.0±0.0	36.3±4.2
Tiros al arco	21.6±3.6	0.2±0.4	19.0±0.0	0.3±0.5	0.0±0.0	34.7±3.3
Intentos de pase	101±5	121±9	118±8	130±14	165±29	195±24
Precisión de pase (%)	52±4	37±4	52±3	37±8	34±2	64±7
Distancia/jugador (m)	1831±56	2042±72	1779±65	1999±56	1849±43	2161±66
Estamina mínima media	3882±258	540±39	4237±182	503±13	3492±395	3096±43
Métricas internas de AutonomoFC						
Error localización P50 / P90 (m)	0.13 / 0.38		0.12 / 0.38		0.11 / 0.37	
Error orientación P50 (°)	0.5		0.5		0.5	
Tiempo de decisión medio / P99 (ms)	0.12 / 0.98		0.10 / 0.81		0.09 / 0.57	

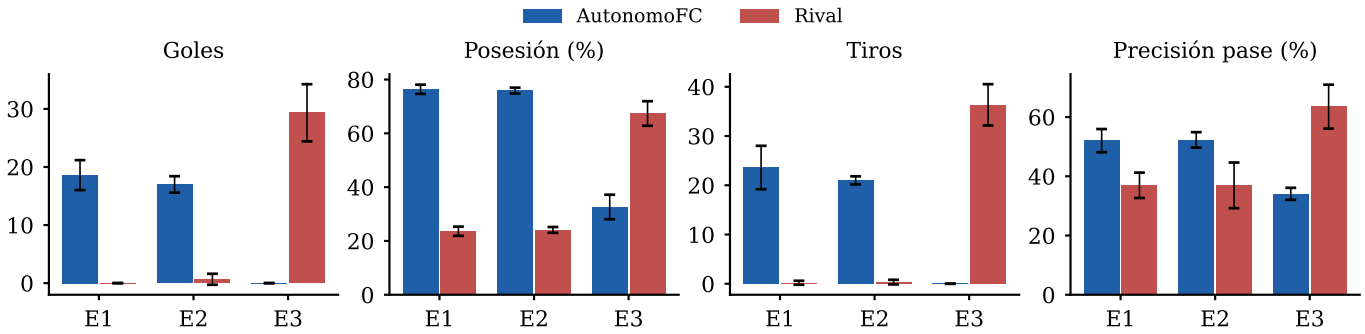


Figura 3. Métricas de equipo por experimento (media ± desviación estándar).

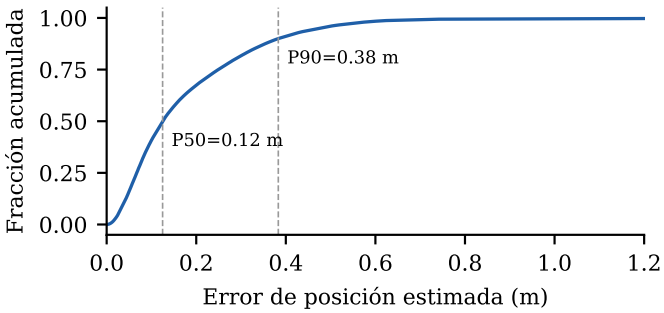


Figura 4. Distribución acumulada del error de auto-localización (E1, 295 889 muestras frente a la posición real del log).

pase cae al 34 % frente al 64 % del rival. El filtro geométrico de pase seguro no anticipa que los rivales se muevan hacia la línea de pase, y la defensa individual, sin línea coordinada, deja espacios entre defensas. En lo interno, la auto-localización tiene un error mediano de 0.12 m y P90 de 0.38 m sobre todas las muestras de E1 (Fig. 4), y cada decisión tarda en promedio 0.1 ms (P99 $<$ 1 ms), muy por debajo del ciclo de 100 ms.

VI. VIDEO DE DEMOSTRACIÓN

El video (3:44 min) muestra, en el monitor oficial, la conexión de los 22 agentes, el saque inicial y fragmentos de E1 y E3 con rótulos de cada experimento. Enlace público: **[pegar aquí el enlace de YouTube o Google Drive]**.

VII. ERRORES Y PROBLEMAS ENCONTRADOS

1. **Configuración creada en un directorio literal "~"**: el servidor, ejecutado en el contenedor sin `HOME` expandido, creó `./~/.rcssserver/`; se resolvió pasando todos los parámetros por línea de comandos.
2. **Parámetro inexistente**: `server::random_seed` no existe en la versión 19; la semilla es `player::random_seed`.
3. **Formato `see` v19**: los jugadores incluyen marcas `k`/`t` (pateó/tackle) que rompían el parser numérico y detenían a los 11 agentes; se filtraron y se protegió el bucle con manejo de excepciones.
4. **Tiempo detenido**: en modos como `free_kick_fault` el contador de ciclos no avanza; el agente solo respondía a ciclos nuevos, no enviaba `(done)` y el servidor abortaba ("Waiting too long for clients"). Ahora se decide en cada `sense_body`.
5. **Arranque sin rival**: en modo sincrónico el `connect_wait` se consumía antes de que conectara el segundo equipo y el partido comenzaba con un solo equipo; se elevó a 100 000 ciclos.
6. **Orientación inestable**: al combinar el ángulo de cabeza de un `see` viejo con el cuello actual, el cuerpo giraba sin control. Se separó el ángulo del cuerpo y se integra el giro comandado.

7. **Doble patada**: tras patear, el modelo seguía viendo el balón en el pie y pateaba otra vez; se actualiza la velocidad del balón con la patada comandada.
8. **Compilación de HELIOS-base**: `librcsc` no detectaba `netinet/in.h` con CMake ni instalaba todos los encabezados; se corrigió `config.h` y se copiaron los encabezados.
9. **Faltas propias (E3)**: 7 `free_kick_fault` (el que saca vuelve a tocar el balón, porque el bloqueo de 15 ciclos se desactiva al pasar a `play_on`) y 2 `back_pass` (el portero ataja pases de compañeros).
10. **Limitaciones abiertas**: el agente ignora los parámetros del tipo heterogéneo asignado (velocidad, margen de pateo) y usa los del tipo 0; la semilla del simulador no se puede fijar en la versión 19 (el parámetro `random_seed` está comentado en el código), así que los partidos no se repiten exactamente y cada uno se trata como una muestra independiente.

VIII. Siguientes pasos

1. Comunicación con `say`/`hear` (posición del balón e intención de pase) para coordinar al interceptor y al receptor.
2. Intercepción con modelo de giro–carrera y velocidad del balón filtrada (Kalman), y pase al espacio con predicción del receptor.
3. Estrategias de defensa colectiva: línea defensiva con trampa de fuera de juego y presión al poseedor por dos jugadores.
4. Jugadas a balón parado (córner, saque de banda) y entrenador online para asignar tipos heterogéneos.
5. Ajuste de parámetros (umbrales de pase/tiro, desplazamientos de la formación) con búsqueda sistemática contra HELIOS-base, y torneo de al menos 20 semillas por configuración con intervalos de confianza.

Referencias

[1] M. Chen *et al.*, "RoboCup Soccer Server: Users Manual," RoboCup Federation, 2003. [En línea]. Disponible: https://rcsoccersim.readthedocs.io

[2] RoboCup Soccer Simulator Maintenance Group, "rcssserver," GitHub. https://github.com/rcsoccersim/rcssserver

[3] RoboCup Soccer Simulator Maintenance Group, "rcssmonitor," GitHub. https://github.com/rcsoccersim/rcssmonitor

[4] H. Akiyama y T. Nakashima, "HELIOS Base: An Open Source Package for the RoboCup Soccer 2D Simulation," en *RoboCup 2013: Robot World Cup XVII*, LNCS 8371, Springer, 2014, pp. 528–535.

APÉNDICE A
CÓDIGO FUENTE

El código completo se entrega también en `codigo_fuente_AutonomoFC.zip`. Estructura: `agente/` (agentes y scripts de ejecución) y `analisis/` (métricas). Ejecución: `./agente/run_match.sh runs/demo player.py AutonomoFC naive.py BaselineChaser`.

Listado 2. agente/sexp.py

```python
"""Parser mínimo de S-expresiones para el protocolo de rcssserver."""


def parse(msg):
    """Convierte '(see 12 ((b) 3 4))' en ['see', '12', [['b'], '3', '4']]."""
    stack = [[]]
    token = ''
    in_str = False
    for ch in msg:
        if in_str:
            token += ch
            if ch == '"':
                in_str = False
            continue
        if ch == '"':
            in_str = True
            token += ch
        elif ch == '(':
            if token:
                stack[-1].append(token)
                token = ''
            stack.append([])
        elif ch == ')':
            if token:
                stack[-1].append(token)
                token = ''
            if len(stack) > 1:
                done = stack.pop()
                stack[-1].append(done)
        elif ch in ' \t\n\x00':
            if token:
                stack[-1].append(token)
                token = ''
        else:
            token += ch
    if token:
        stack[-1].append(token)
    root = stack[0]
    return root[0] if len(root) == 1 else root
```

Listado 3. agente/world.py

```python
"""Modelo del mundo: auto-localización por banderas, balón, compañeros y rivales.

Todas las coordenadas internas están en el marco del equipo: el agente siempre
ataca hacia +x (si juega por la derecha se invierten las banderas).
"""
import math
from sexp import parse

# ---------------------------------------------------------------- banderas
FLAGS = {
    'g l': (-52.5, 0), 'g r': (52.5, 0),
    'f c': (0, 0), 'f c t': (0, -34), 'f c b': (0, 34),
    'f l t': (-52.5, -34), 'f l b': (-52.5, 34),
    'f r t': (52.5, -34), 'f r b': (52.5, 34),
    'f p l t': (-36, -20.16), 'f p l c': (-36, 0), 'f p l b': (-36, 20.16),
    'f p r t': (36, -20.16), 'f p r c': (36, 0), 'f p r b': (36, 20.16),
    'f g l t': (-52.5, -7.01), 'f g l b': (-52.5, 7.01),
    'f g r t': (52.5, -7.01), 'f g r b': (52.5, 7.01),
    'f t 0': (0, -39), 'f b 0': (0, 39), 'f l 0': (-57.5, 0), 'f r 0': (57.5, 0),
}
for d in (10, 20, 30, 40, 50):
    FLAGS['f t l %d' % d] = (-d, -39)
    FLAGS['f t r %d' % d] = (d, -39)
    FLAGS['f b l %d' % d] = (-d, 39)
    FLAGS['f b r %d' % d] = (d, 39)
for d in (10, 20, 30):
    FLAGS['f l t %d' % d] = (-57.5, -d)
    FLAGS['f l b %d' % d] = (-57.5, d)
    FLAGS['f r t %d' % d] = (57.5, -d)
    FLAGS['f r b %d' % d] = (57.5, d)


def norm_angle(a):
    while a > 180:
        a -= 360
    while a < -180:
        a += 360
    return a


def dist(a, b):
    return math.hypot(a[0] - b[0], a[1] - b[1])


def polar(d, ang_deg):
    r = math.radians(ang_deg)
    return d * math.cos(r), d * math.sin(r)


class Obj:
    def __init__(self, pos, t, vel=(0.0, 0.0), unum=None, goalie=False):
        self.pos = pos
        self.t = t
        self.vel = vel
        self.unum = unum
        self.goalie = goalie


class World:
    def __init__(self, team):
        self.team = team
        self.side = 'l'
        self.unum = 0
        self.playmode = 'before_kick_off'
        self.time = 0
        self.see_time = -1
        # estado propio
        self.pos = (0.0, 0.0)
        self.head = 0.0          # ángulo global de la cabeza (grados)
        self.neck = 0.0          # ángulo cuello relativo al cuerpo
        self.stamina = 8000.0
        self.speed = 0.0
        self.pos_conf = 0        # nº de banderas usadas en la última estimación
        self.speed_dir = 0.0
        self._body = 0.0         # ángulo global del cuerpo
        self.pending_turn = 0.0  # giro efectivo del último comando turn
        # objetos
        self.ball = None
        self.ball_prev = None
        self.mates = []
        self.opps = []
        self.score = [0, 0]      # [nuestro, rival]

    # ------------------------------------------------------- derivados
    @property
    def body(self):
        return self._body

    def flag_table(self):
        if self.side == 'l':
            return FLAGS
        return {k: (-v[0], -v[1]) for k, v in FLAGS.items()}

    def ball_age(self):
        return self.time - self.ball.t if self.ball else 999

    def ball_pred(self, n):
        """Posición del balón dentro de n ciclos (decaimiento 0.94)."""
        if not self.ball:
            return None
        (x, y), (vx, vy) = self.ball.pos, self.ball.vel
        k = (1 - 0.94 ** n) / 0.06
        return (x + vx * k, y + vy * k)
```

```
104
105     def advance(self):
106         """Estimación por odometría cuando no hubo 'see' en este
        ciclo."""
107         self._body = norm_angle(self._body + self.pending_turn)
108         self.head = norm_angle(self._body + self.neck)
109         a = math.radians(self.head + self.speed_dir)
110         self.pos = (self.pos[0] + self.speed * math.cos(a),
111                     self.pos[1] + self.speed * math.sin(a))
112         if self.ball:
113             (x, y), (vx, vy) = self.ball.pos, self.ball.vel
114             self.ball.pos = (x + vx, y + vy)
115             self.ball.vel = (vx * 0.94, vy * 0.94)
116
117     # ------------------------------------------------------------
        mensajes
118     def handle(self, raw):
119         msg = parse(raw)
120         if not msg or not isinstance(msg, list):
121             return None
122         kind = msg[0]
123         if kind == 'init':
124             self.side, self.unum, self.playmode = msg[1], int(
        msg[2]), msg[3]
125         elif kind == 'sense_body':
126             self._sense_body(msg)
127         elif kind == 'see':
128             self._see(msg)
129         elif kind == 'hear':
130             self._hear(msg)
131         return kind
132
133     def _sense_body(self, msg):
134         self.time = int(msg[1])
135         for item in msg[2:]:
136             if not isinstance(item, list):
137                 continue
138             if item[0] == 'stamina':
139                 self.stamina = float(item[1])
140             elif item[0] == 'speed':
141                 self.speed = float(item[1])
142                 self.speed_dir = float(item[2]) if len(item) > 2
          else 0.0
143             elif item[0] == 'head_angle':
144                 self.neck = float(item[1])
145
146     def _hear(self, msg):
147         t, sender = int(msg[1]), msg[2]
148         if sender == 'referee':
149             pm = msg[3]
150             self.playmode = pm
151             if pm.startswith('goal_'):
152                 parts = pm.split('_')
153                 if parts[1] == self.side:
154                     self.score[0] = int(parts[2])
155                 else:
156                     self.score[1] = int(parts[2])
157
158     def _see(self, msg):
159         t = int(msg[1])
160         self.see_time = t
161         self.time = max(self.time, t)
162         flags = self.flag_table()
163         seen_flags = []
164         raw_ball = None
165         raw_players = []
166         for obj in msg[2:]:
167             name = obj[0]
168             vals = []
169             for v in obj[1:]:
170                 try:
171                     vals.append(float(v))
172                 except (TypeError, ValueError):
173                     pass                                # marcas 'k'/'t
        ' (patada/tackle) en v19
174             if len(vals) < 2:
175                 continue
176             key = ' '.join(name)
177             if name[0] in ('f', 'g') and key in flags:
178                 seen_flags.append((flags[key], vals[0], vals[1])
        )
179             elif name[0] in ('b', 'B'):
180                 raw_ball = vals
181             elif name[0] in ('p', 'P'):
182                 raw_players.append((name, vals))
183         self._localize(seen_flags)
184         if raw_ball:
185             self._update_ball(raw_ball, t)
186         self._update_players(raw_players, t)
187
188     # ------------------------------------------------ auto-
        localización
189     def _localize(self, seen):
190         """Triangulación por pares de banderas: estima ángulo de
        cabeza y posición.
191
192         Para dos banderas A,B con vectores relativos rA,rB (
        marco de la cabeza):
193         theta = atan2(B-A) - atan2(rB-rA);  p = A - R(theta) rA.
194         """
195         if len(seen) < 2:
196             return
197         seen.sort(key=lambda s: s[1])
198         seen = seen[:8]                             # las más
        cercanas = menos ruido
199         sx = sy = 0.0
200         for i in range(len(seen)):
201             for j in range(i + 1, len(seen)):
202                 (A, dA, aA), (B, dB, aB) = seen[i], seen[j]
203                 rA, rB = polar(dA, aA), polar(dB, aB)
204                 gx, gy = B[0] - A[0], B[1] - A[1]
205                 lx, ly = rB[0] - rA[0], rB[1] - rA[1]
206                 if math.hypot(lx, ly) < 5:        # par mal
        condicionado
207                     continue
208                 th = math.atan2(gy, gx) - math.atan2(ly, lx)
209                 sx += math.cos(th)
210                 sy += math.sin(th)
211         if sx == 0 and sy == 0:
212             return
213         theta = math.degrees(math.atan2(sy, sx))
214         px = py = wsum = 0.0
215         for (F, d, a) in seen:
216             vx, vy = polar(d, a + theta)
217             w = 1.0 / (1.0 + d)                     # pondera
        banderas cercanas
218             px += (F[0] - vx) * w
219             py += (F[1] - vy) * w
220             wsum += w
221         self.head = theta
222         self._body = norm_angle(theta - self.neck)
223         self.pos = (px / wsum, py / wsum)
224         self.pos_conf = len(seen)
225
226     def to_global(self, d, a):
227         vx, vy = polar(d, a + self.head)
228         return (self.pos[0] + vx, self.pos[1] + vy)
229
230     def _update_ball(self, vals, t):
231         pos = self.to_global(vals[0], vals[1])
232         vel = (0.0, 0.0)
233         if len(vals) >= 4:
234             # distchg/dirchg -> velocidad relativa aproximada
235             d, a, dd, da = vals[:4]
236             ar = math.radians(a + self.head)
237             vr = dd
238             vt = math.radians(da) * d
239             vel = (vr * math.cos(ar) - vt * math.sin(ar),
240                    vr * math.sin(ar) + vt * math.cos(ar))
241         elif self.ball and 0 < t - self.ball.t <= 3:
242             dt = t - self.ball.t
243             vel = ((pos[0] - self.ball.pos[0]) / dt, (pos[1] -
        self.ball.pos[1]) / dt)
244         self.ball_prev = self.ball
245         self.ball = Obj(pos, t, vel)
246
247     def _update_players(self, raws, t):
248         mates, opps = [], []
249         for name, vals in raws:
250             pos = self.to_global(vals[0], vals[1])
251             team = name[1].strip('"') if len(name) > 1 else None
252             unum = int(name[2]) if len(name) > 2 else None
253             goalie = len(name) > 3 and name[3] == 'goalie'
254             o = Obj(pos, t, unum=unum, goalie=goalie)
255             if team is None:
256                 # jugador lejano sin equipo: se ignora
257                 continue
258             (mates if team == self.team else opps).append(o)
259         self.mates, self.opps = mates, opps
260
261     # ------------------------------------------------------------
        utilidades
262     def offside_line(self):
263         xs = sorted([o.pos[0] for o in self.opps], reverse=True)
264         line = xs[1] if len(xs) >= 2 else 0.0
265         if self.ball:
266             line = max(line, self.ball.pos[0])
267         return max(line, 0.0)
268
269     def nearest_opp(self, p):
270         if not self.opps:
271             return None, 99.0
272         o = min(self.opps, key=lambda o: dist(o.pos, p))
273         return o, dist(o.pos, p)
```

```
274
275     def our_set_piece(self):
276         pm = self.playmode
277         for k in ('kick_off_', 'kick_in_', 'free_kick_', '
        corner_kick_',
278                   'goal_kick_', 'offside_', 'indirect_free_kick_
        ', 'foul_charge_',
279                   'back_pass_', 'free_kick_fault_', '
        goalie_catch_ball_'):
280             if pm.startswith(k):
281                 letter = pm[len(k):len(k) + 1]
282                 # offside_X / back_pass_X / foul_charge_X: falta
         DEL equipo X
283                 if k in ('offside_', 'back_pass_', 'foul_charge_
        ', 'free_kick_fault_'):
284                     return letter != self.side
285                 return letter == self.side
286         return False
287
288     def their_set_piece(self):
289         return self.playmode != 'play_on' and not self.
        our_set_piece() and \
290             not self.playmode.startswith(('before_kick_off', '
        goal_', 'time_over', 'half_time'))
```

Listado 4. agente/player.py

```
1  """Agente autónomo 'AutonomoFC' para RoboCup Soccer Simulation 2
       D (rcssserver).
2
3  Arquitectura percepción -> modelo del mundo -> decisión -> acció
       n, un proceso por
4  jugador comunicándose por UDP con el servidor.
5
6  Estrategias:
7    * Formación 4-3-3 dinámica: la posición base se desplaza con
         el balón.
8    * Rol de interceptor elegido por tiempo estimado de llegada al
          balón.
9    * Árbol de decisión con balón: tiro > pase seguro > conducción
          > despeje.
10   * Evaluación de pases con riesgo de intercepción por rivales.
11   * Línea de fuera de juego estimada para delanteros.
12   * Portero: cobertura del ángulo de tiro, atajada y saque.
13   * Gestión de estamina y escaneo con el cuello.
14
15 Uso: python3 player.py --team AutonomoFC --unum 1 [--goalie] [--
       host 127.0.0.1]
16 """
17 import argparse
18 import json
19 import math
20 import os
21 import random
22 import socket
23 import time
24
25 from world import World, dist, norm_angle
26
27 KICKABLE = 0.3 + 0.7 + 0.085        # player_size +
       kickable_margin + ball_size
28 CATCHABLE = 1.2
29 GOAL_X = 52.5
30
31 # Formación 4-3-3 (marco del equipo, balón en el centro del
       campo)
32 FORMATION = {
33     1: (-50, 0),
34     2: (-30, -22), 3: (-33, -8), 4: (-33, 8), 5: (-30, 22),
35     6: (-15, -14), 7: (-18, 0), 8: (-15, 14),
36     9: (3, -22), 10: (6, 0), 11: (3, 22),
37 }
38 KICKOFF = {
39     1: (-50, 0),
40     2: (-35, -20), 3: (-37, -7), 4: (-37, 7), 5: (-35, 20),
41     6: (-22, -12), 7: (-24, 0), 8: (-22, 12),
42     9: (-6, -20), 10: (-0.6, 0), 11: (-6, 20),
43 }
44
45
46 class Agent:
47     def __init__(self, team, unum, goalie, host, port, log_dir,
        version=19, synch=False):
48         self.synch = synch
49         self.team, self.want_unum, self.goalie = team, unum,
       goalie
50         self.wm = World(team)
51         self.sock = socket.socket(socket.AF_INET, socket.
       SOCK_DGRAM)
52         self.sock.settimeout(0.5)
53         self.addr = (host, port)
54         init = '(init %s (version %d)%s)' % (team, version, ' (
       goalie)' if goalie else '')
55         self.sock.sendto(init.encode(), self.addr)
56         self.last_cycle = -1
57         self.moved_mode = None
58         self.set_piece_kicked_at = -100
59         self.scan_dir = 1
60         self.log = []
61         self.log_dir = log_dir
62         self.decision = ''
63
64 #
    ------------------------------------------------------------
     red
65     def send(self, cmd):
66         self.sock.sendto((cmd + '\x00').encode(), self.addr)
67
68     def run(self):
69         while True:
70             try:
71                 data, addr = self.sock.recvfrom(8192)
72             except socket.timeout:
73                 if self.wm.playmode == 'time_over':
74                     break
75                 continue
76             self.addr = addr                        # el servidor
        responde desde otro puerto
77             raw = data.decode(errors='ignore').strip('\x00')
78             if raw.startswith('(error'):
79                 continue
80             try:
81                 kind = self.wm.handle(raw)
82             except Exception as e:              # un mensaje
       malformado no detiene al agente
83                 self.errors = getattr(self, 'errors', 0) + 1
84                 continue
85             if kind == 'init':
86                 self.send('(synch_see)')
87                 self.send('(change_view normal)')
88             elif kind == 'sense_body':
89                 self.drain()
90                 # se decide una vez por sense_body (también en
       tiempo detenido)
91                 if self.wm.see_time < self.wm.time and self.
       last_cycle >= 0:
92                     self.wm.advance()
93                 self.last_cycle = self.wm.time
94                 t0 = time.perf_counter()
95                 self.step()
96                 if self.synch:
97                     self.send('(done)')
98                 ms = (time.perf_counter() - t0) * 1000
99                 w = self.wm
100                self.log.append([w.time, round(w.pos[0], 2),
       round(w.pos[1], 2),
101                                 round(w.body, 1), w.pos_conf,
       self.decision,
102                                 round(ms, 3), round(w.stamina)
       ])
103            if self.wm.playmode == 'time_over':
104                break
105        self.save_log()
106
107    def drain(self, wait=0.004):
108        """Procesa mensajes que llegan junto al sense_body (p.ej
       . el 'see' del ciclo)."""
109        self.sock.settimeout(wait)
110        try:
111            while True:
112                data, addr = self.sock.recvfrom(8192)
113                raw = data.decode(errors='ignore').strip('\x00')
114                try:
115                    self.wm.handle(raw)
116                except Exception:
117                    pass
118        except (socket.timeout, BlockingIOError):
119            pass
120        finally:
121            self.sock.settimeout(0.5)
122
123    def save_log(self):
124        if not self.log_dir:
125            return
126        os.makedirs(self.log_dir, exist_ok=True)
127        fn = os.path.join(self.log_dir, '%s_%s_%d.json' % (self
       .team, self.wm.side, self.wm.unum))
128        with open(fn, 'w') as f:
129            json.dump({'team': self.team, 'side': self.wm.side,
       'unum': self.wm.unum,
130                       'cols': ['t', 'x', 'y', 'body', 'nflags',
        'dec', 'ms', 'stamina'],
131                       'rows': self.log}, f)
```

```
    # ------------------------------------------------------- primitivas
    def turn_to(self, point):
        w = self.wm
        ang = math.degrees(math.atan2(point[1] - w.pos[1], point[0] - w.pos[0]))
        rel = norm_angle(ang - w.body)
        moment = max(-180, min(180, rel * (1 + 5 * w.speed)))  # compensa inercia
        w.pending_turn = moment / (1 + 5 * w.speed)
        self.send('(turn %.1f)' % moment)

    def go_to(self, point, power=100, tol=1.0):
        w = self.wm
        d = dist(w.pos, point)
        if d < tol:
            return False
        ang = math.degrees(math.atan2(point[1] - w.pos[1], point[0] - w.pos[0]))
        rel = norm_angle(ang - w.body)
        if abs(rel) > (25 if d > 5 else 40):
            self.turn_to(point)
        else:
            if w.stamina < 3000:
                power = min(power, 40)
            elif w.stamina < 4500:
                power = min(power, 70)
            self.send('(dash %d %.1f)' % (power, rel if abs(rel) > 8 else 0))
        return True

    def kick_to(self, target, speed):
        """Patea para que el balón salga hacia target con velocidad 'speed'."""
        w = self.wm
        b = w.ball
        bdir = math.degrees(math.atan2(b.pos[1] - w.pos[1], b.pos[0] - w.pos[0]))
        bd = dist(w.pos, b.pos)
        dir_diff = abs(norm_angle(bdir - w.body))
        eff = 0.027 * (1 - 0.25 * dir_diff / 180 - 0.25 * max(0, bd - 0.385) / 0.7)
        ang = math.atan2(target[1] - b.pos[1], target[0] - b.pos[0])
        speed = min(speed, 3.0)
        ax = speed * math.cos(ang) - b.vel[0]
        ay = speed * math.sin(ang) - b.vel[1]
        acc = math.hypot(ax, ay)
        power = min(100, acc / max(eff, 1e-3))
        kdir = norm_angle(math.degrees(math.atan2(ay, ax)) - w.body)
        self.send('(kick %.1f %.1f)' % (power, kdir))
        # actualización optimista del modelo: el balón sale con la velocidad pedida
        got = min(1.0, power * eff / max(acc, 1e-6))
        b.vel = (b.vel[0] + ax * got, b.vel[1] + ay * got)

    def look(self):
        """Cuello: sigue el balón si se conoce, si no barre el campo."""
        w = self.wm
        if w.ball and w.ball_age() <= 2:
            ang = math.degrees(math.atan2(w.ball.pos[1] - w.pos[1], w.ball.pos[0] - w.pos[0]))
            target = max(-90, min(90, norm_angle(ang - w.body)))
        else:
            target = w.neck + 60 * self.scan_dir
            if abs(target) > 90:
                self.scan_dir *= -1
                target = w.neck + 60 * self.scan_dir
        self.send('(turn_neck %.1f)' % (norm_angle(target - w.neck)))

    # ------------------------------------------------------- decisión
    def step(self):
        w = self.wm
        w.pending_turn = 0.0
        pm = w.playmode
        if pm == 'time_over':
            return
        if pm.startswith(('before_kick_off', 'goal_', 'half_time')):
            if self.moved_mode != (pm, w.time // 50):
                self.moved_mode = (pm, w.time // 50)
                x, y = KICKOFF[w.unum]
                self.send('(move %.1f %.1f)' % (x, y))
            self.decision = 'KICKOFF_POS'
            self.send('(turn 30)') if not w.ball else self.turn_to(w.ball.pos)
            return
        if not w.ball or w.ball_age() > 4:
            self.decision = 'SEARCH'
            self.wm.pending_turn = 60 / (1 + 5 * w.speed)
            self.send('(turn 60)')
            return

        self.look()
        if self.goalie:
            self.goalie_step()
            return

        bd = dist(w.pos, w.ball.pos)
        if bd <= KICKABLE:
            self.with_ball()
        elif self.i_am_chaser():
            self.decision = 'INTERCEPT'
            self.go_to(self.intercept_point(), 100, tol=0.4)
        else:
            self.positioning()

    # ------------------------------------------ selección de interceptor
    def reach_cycles(self, p):
        return max(0.0, dist(self.wm.pos, p) - KICKABLE) / 0.9

    def intercept_point(self):
        w = self.wm
        for n in range(0, 40):
            bp = w.ball_pred(n)
            if self.reach_cycles(bp) <= n:
                return bp
        return w.ball_pred(40)

    def i_am_chaser(self):
        w = self.wm
        # tras sacar un balón parado no puedo volver a tocarlo (doble toque)
        if w.time - self.set_piece_kicked_at < 15 and w.playmode != 'play_on':
            return False
        if w.their_set_piece():
            return False
        my_t = self.reach_cycles(self.intercept_point())
        for m in w.mates:
            if m.goalie:
                continue
            if max(0.0, dist(m.pos, w.ball.pos) - KICKABLE) / 0.9 < my_t - 1.5:
                return False
        return True

    # ------------------------------------------------------- con el balón
    def with_ball(self):
        w = self.wm
        me = w.pos
        set_piece = w.our_set_piece()
        goal_d = dist(me, (GOAL_X, 0))
        _, opp_d = w.nearest_opp(me)

        # 1) Tiro
        if goal_d < 22 and not set_piece:
            self.decision = 'SHOOT'
            self.kick_to(self.shoot_target(), 3.0)
            return
        # 2) Pase
        target, score = self.best_pass()
        pressured = opp_d < 5
        if target and (set_piece or pressured or score > 12):
            self.decision = 'PASS'
            d = dist(me, target)
            self.kick_to(target, min(3.0, 0.9 + 0.055 * d))
            if set_piece:
                self.set_piece_kicked_at = w.time
            return
        # 3) Despeje en zona defensiva bajo presión o saque sin opciones
        if set_piece or (me[0] < -25 and pressured):
            self.decision = 'CLEAR'
            side = 1 if me[1] >= 0 else -1
            self.kick_to((me[0] + 35, side * 25), 3.0)
            if set_piece:
                self.set_piece_kicked_at = w.time
            return
        # 4) Conducción
        self.decision = 'DRIBBLE'
        self.kick_to(self.dribble_target(), 0.85)

    def shoot_target(self):
        w = self.wm
        gk = [o for o in w.opps if o.goalie] or \
             sorted(w.opps, key=lambda o: dist(o.pos, (GOAL_X,
```

```
        0)))[:1]
        gy = gk[0].pos[1] if gk else 0.0
        y = -5.8 if gy > 0 else 5.8
        if not gk:
            y = random.choice((-5.8, 5.8))
        return (GOAL_X, y)

    def dribble_target(self):
        w = self.wm
        me = w.pos
        base = math.atan2(-me[1] * 0.3, GOAL_X - me[0])
        best, best_s = None, -1e9
        for off in (-60, -40, -20, 0, 20, 40, 60):
            a = base + math.radians(off)
            p = (me[0] + 5 * math.cos(a), me[1] + 5 * math.sin(a
        ))
            if abs(p[1]) > 32 or p[0] > 51:
                continue
            _, od = w.nearest_opp(p)
            s = p[0] * 0.3 + min(od, 8) * 1.2 - abs(off) * 0.02
            if s > best_s:
                best, best_s = p, s
        return best or (me[0] + 5, me[1])

    def pass_safe(self, src, dst):
        """Riesgo de intercepción: un rival que llega antes al
         segmento."""
        L = dist(src, dst)
        if L < 1:
            return False
        ux, uy = (dst[0] - src[0]) / L, (dst[1] - src[1]) / L
        for o in self.wm.opps:
            rx, ry = o.pos[0] - src[0], o.pos[1] - src[1]
            proj = rx * ux + ry * uy
            if proj < 0 or proj > L + 2:
                continue
            perp = abs(-rx * uy + ry * ux)
            if perp < 1.5 + 0.35 * proj:
                return False
        return True

    def best_pass(self):
        w = self.wm
        me = w.pos
        best, best_s = None, -1e9
        for m in w.mates:
            if m.unum == w.unum:
                continue
            d = dist(me, m.pos)
            if d < 5 or d > 32:
                continue
            # pase al hueco: un poco adelante del receptor
            tgt = (min(m.pos[0] + 2.0, 50), m.pos[1])
            if not self.pass_safe(me, tgt):
                continue
            _, od = w.nearest_opp(tgt)
            s = (tgt[0] - me[0]) + min(od, 10) * 1.0 - 0.1 * d
            if m.goalie or tgt[0] < -36:
                s -= 15
            if s > best_s:
                best, best_s = tgt, s
        return best, best_s

    # ------------------------------------------------------ sin
      balón
    def home_position(self):
        w = self.wm
        bx, by = w.ball.pos
        fx, fy = FORMATION[w.unum]
        x = fx + 0.55 * bx
        y = fy * 0.9 + 0.25 * by
        if w.unum >= 9:                              # delanteros:
         respetar fuera de juego
            x = min(x, w.offside_line() - 1.0)
        if w.unum in (2, 3, 4, 5):
            x = max(-46, min(x, bx + 5))             # defensas no
         pasan del balón
        return (max(-48, min(48, x)), max(-32, min(32, y)))

    def positioning(self):
        w = self.wm
        home = self.home_position()
        far = dist(w.pos, home)
        self.decision = 'POSITION'
        # marcaje sencillo: defensas/medios cubren al rival más
         peligroso cercano
        if w.ball.pos[0] < -10 and w.unum in (2, 3, 4, 5, 6, 7,
        8):
            cands = [o for o in w.opps if dist(o.pos, home) < 9
         and o.pos[0] < 0]
            if cands:
                o = min(cands, key=lambda o: dist(o.pos, (-
         GOAL_X, 0)))
                gx, gy = -GOAL_X - o.pos[0], -o.pos[1]
                n = math.hypot(gx, gy) or 1
                home = (o.pos[0] + 1.5 * gx / n, o.pos[1] + 1.5
        * gy / n)
                self.decision = 'MARK'
        power = 100 if far > 8 else 60
        if not self.go_to(home, power, tol=1.5):
            self.turn_to(w.ball.pos)

    # ------------------------------------------------------
      portero
    def goalie_step(self):
        w = self.wm
        b = w.ball.pos
        bd = dist(w.pos, b)
        in_box = b[0] < -36 and abs(b[1]) < 20.16
        if w.our_set_piece():
            if bd <= KICKABLE:
                self.decision = 'GK_KICK'
                tgt, _ = self.best_pass()
                side = 1 if w.pos[1] >= 0 else -1
                self.kick_to(tgt or (-15, side * 25), 3.0)
                self.set_piece_kicked_at = w.time
            elif w.playmode.startswith('goal_kick') and w.time -
         self.set_piece_kicked_at > 15:
                self.decision = 'GK_FETCH'
                self.go_to(b, 100, tol=0.5)
            else:
                self.decision = 'GK_WAIT'
                self.turn_to(b)
            return
        if in_box and bd < CATCHABLE and w.playmode == 'play_on'
        :
            ang = math.degrees(math.atan2(b[1] - w.pos[1], b[0]
         - w.pos[0]))
            self.decision = 'GK_CATCH'
            self.send('(catch %.1f)' % norm_angle(ang - w.body)
        )
            return
        if in_box and self.i_am_chaser() and b[0] < -40:
            self.decision = 'GK_INTERCEPT'
            self.go_to(self.intercept_point(), 100, tol=0.4)
            return
        # cobertura del ángulo: sobre la bisectriz balón-arco
        g = (-GOAL_X, 0)
        vx, vy = b[0] - g[0], b[1] - g[1]
        n = math.hypot(vx, vy) or 1
        r = max(1.5, min(5.0, 0.12 * n))
        tgt = (g[0] + r * vx / n, max(-6.5, min(6.5, g[1] + r *
         vy / n)))
        self.decision = 'GK_POSITION'
        if not self.go_to(tgt, 80, tol=0.7):
            self.turn_to(b)


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('--team', default='AutonomoFC')
    ap.add_argument('--unum', type=int, default=1)
    ap.add_argument('--goalie', action='store_true')
    ap.add_argument('--host', default='127.0.0.1')
    ap.add_argument('--port', type=int, default=6000)
    ap.add_argument('--log-dir', default='')
    ap.add_argument('--synch', action='store_true')
    a = ap.parse_args()
    Agent(a.team, a.unum, a.goalie, a.host, a.port, a.log_dir,
     synch=a.synch).run()


if __name__ == '__main__':
    main()
```

Listado 5. agente/naive.py

```
"""Equipo base 'BaselineChaser' (línea base de comparación).

Comportamiento reactivo clásico de 'kiddy soccer': todos los
      jugadores de
campo corren hacia el balón y, si lo alcanzan, patean a máxima
      potencia hacia
el arco rival. Sin formación, sin pases, sin roles. Reutiliza el
       modelo del
mundo para que la comparación aísle la capa de decisión.
"""
import argparse, math
from player import Agent, KICKOFF, GOAL_X
from world import dist


class Chaser(Agent):
    def step(self):
```

```
        w = self.wm
        w.pending_turn = 0.0
        pm = w.playmode
        if pm.startswith(('before_kick_off', 'goal_', 'half_time')):
            if self.moved_mode != (pm, w.time // 50):
                self.moved_mode = (pm, w.time // 50)
                self.send('(move %.1f %.1f)' % KICKOFF[w.unum])
            self.decision = 'KICKOFF_POS'
            return
        if not w.ball or w.ball_age() > 4:
            self.decision = 'SEARCH'
            self.send('(turn 60)')
            return
        if w.their_set_piece():
            self.decision = 'WAIT'
            self.turn_to(w.ball.pos)
            return
        if self.goalie and (w.ball.pos[0] > -36 or abs(w.ball.pos[1]) > 20):
            self.decision = 'GK_POSITION'
            if not self.go_to((-50, max(-5, min(5, w.ball.pos[1] * 0.2))), 60):
                self.turn_to(w.ball.pos)
            return
        if dist(w.pos, w.ball.pos) <= 1.085:
            self.decision = 'KICK_GOAL'
            self.kick_to((GOAL_X, 0), 3.0)
        else:
            self.decision = 'CHASE'
            self.go_to(w.ball.pos, 100, tol=0.3)


if __name__ == '__main__':
    ap = argparse.ArgumentParser()
    ap.add_argument('--team', default='BaselineChaser')
    ap.add_argument('--unum', type=int, default=1)
    ap.add_argument('--goalie', action='store_true')
    ap.add_argument('--host', default='127.0.0.1')
    ap.add_argument('--port', type=int, default=6000)
    ap.add_argument('--log-dir', default='')
    ap.add_argument('--synch', action='store_true')
    a = ap.parse_args()
    Chaser(a.team, a.unum, a.goalie, a.host, a.port, a.log_dir, synch=a.synch).run()
```

Listado 6. agente/start_team.sh

```
#!/usr/bin/env bash
# Lanza 11 jugadores: ./start_team.sh <player.py|naive.py> <EQUIPO> <log_dir> [--synch] [puerto]
SCRIPT=${1:-player.py}; TEAM=${2:-AutonomoFC}; LOG=${3:-logs}; EXTRA=$4; PORT=${5:-6000}
DIR=$(cd "$(dirname "$0")" && pwd)
python3 "$DIR/$SCRIPT" --team "$TEAM" --unum 1 --goalie --log-dir "$LOG" --port $PORT $EXTRA &
sleep 0.3
for i in $(seq 2 11); do
  python3 "$DIR/$SCRIPT" --team "$TEAM" --unum $i --log-dir "$LOG" --port $PORT $EXTRA &
  sleep 0.1
done
wait
```

Listado 7. agente/run_match.sh

```
#!/usr/bin/env bash
# Partido en modo sincrónico (rápido) con logs.
# ./run_match.sh <out_dir> <left_script> <left_team> <right_script> <right_team> [half_time_s] [seed] [port]
# Un "script" puede ser player.py, naive.py o 'helios' (equipo helios-base compilado).
OUT=$1; LS=$2; LT=$3; RS=$4; RT=$5; HT=${6:-300}; SEED=${7:--1}; PORT=${8:-6000}
SYNCH=${SYNCH:-true}
AG=$(cd "$(dirname "$0")" && pwd)
SRV=${SRV:-/home/claude/proyecto/rcssserver/build/rcssserver}
HELIOS=${HELIOS:-/home/claude/proyecto/ext/helios-base/build/bin}
mkdir -p "$OUT"; cd "$OUT"
$SRV server::synch_mode=$SYNCH server::auto_mode=true server::half_time=$HT \
  server::port=$PORT server::coach_port=$((PORT+1)) server::olcoach_port=$((PORT+2)) \
  server::nr_extra_halfs=0 server::penalty_shoot_outs=false \
  server::game_log_dir=./ server::text_log_dir=./ server::game_log_dated=false \
  server::text_log_dated=false server::game_log_fixed=true server::text_log_fixed=true \
  server::game_log_fixed_name=match server::text_log_fixed_name=match \
  server::kick_off_wait=30 server::connect_wait=100000 server::game_over_wait=20 \
  player::random_seed=$SEED > server.out 2>&1 &
SPID=$!
sleep 1
launch() {  # script team side
  F=""; [ "$SYNCH" = "true" ] && F="--synch"
  if [ "$1" = "helios" ]; then
    (cd "$HELIOS" && LD_LIBRARY_PATH=/home/claude/proyecto/ext/local/lib ./start.sh -t "$2" -p $PORT -P $((PORT+2)) > "$OUT/$3.out" 2>&1 &)
  else
    "$AG/start_team.sh" "$1" "$2" "$OUT/agents" "$F" $PORT > "$OUT/$3.out" 2>&1 &
  fi
}
launch "$LS" "$LT" left
sleep 2
launch "$RS" "$RT" right
wait $SPID
sleep 2
pkill -f "port $PORT" 2>/dev/null
grep Score server.out
```

Listado 8. analisis/metrics.py

```
"""Cálculo de métricas a partir del log del servidor (.rcg) y de los logs de agentes.

Uso: python3 metrics.py <carpeta_partido> [--json salida.json]
"""
import glob
import json
import math
import os
import re
import sys
from collections import Counter, defaultdict

PLAYER_RE = re.compile(
    r'\((l|r) (\d+)\) (\d+) (\S+) (\S+) (\S+) (\S+) (\S+) (\S+) (\S+) (\S+) (\S+)\)')
BALL_RE = re.compile(r'\(b\) (\S+) (\S+) (\S+) (\S+)\)')
STAM_RE = re.compile(r'\(s (\S+) (\S+) (\S+) (\S+)\)')


def parse_rcg(path):
    frames, modes, teams = [], [], None
    for line in open(path, errors='ignore'):
        if line.startswith('(show '):
            t = int(line.split()[1])
            b = BALL_RE.search(line)
            ball = tuple(map(float, b.groups()))
            players = {}
            for m in PLAYER_RE.finditer(line):
                side, unum = m.group(1), int(m.group(2))
                x, y, vx, vy, body, neck = map(float, m.group(5, 6, 7, 8, 9, 10))
                st = STAM_RE.search(m.group(11))
                stamina = float(st.group(1)) if st else 0.0
                counts = list(map(int, m.group(12).split()))
                players[(side, unum)] = dict(x=x, y=y, vx=vx, vy=vy, body=body, neck=neck,
                                             stamina=stamina, kick=counts[0], dash=counts[1],
                                             state=int(m.group(4), 16))
            frames.append((t, ball, players))
        elif line.startswith('(playmode '):
            _, t, mode = line.strip('()\n').split()
            modes.append((int(t), mode))
        elif line.startswith('(team '):
            p = line.strip('()\n').split()
            teams = dict(left=p[2], right=p[3], score_l=int(p[4]), score_r=int(p[5]))
    return frames, modes, teams


def team_frame(side, x, y):
    return (x, y) if side == 'l' else (-x, -y)


def match_metrics(folder):
    frames, modes, teams = parse_rcg(os.path.join(folder, 'match.rcg'))
    # modo de juego vigente en cada frame
    mode_at = []
    mi, cur = 0, 'before_kick_off'
    for (t, ball, pl) in frames:
        while mi < len(modes) and modes[mi][0] <= t:
            cur = modes[mi][1]
            mi += 1
```

```
        mode_at.append(cur)

    res = {s: defaultdict(float) for s in ('l', 'r')}
    poss_cycles = Counter()
    last_holder = None
    kicks = []                                  # (t, side, unum, ball_after)
    dist_run = defaultdict(float)
    prev = None
    min_stamina = defaultdict(lambda: 9e9)
    for i, (t, ball, pl) in enumerate(frames):
        bx, by = ball[0], ball[1]
        play = mode_at[i] == 'play_on'
        # posesión (último jugador en zona de control del balón)
        near = [(math.hypot(p['x'] - bx, p['y'] - by), k) for k, p in pl.items()]
        if near:
            d, k = min(near)
            if d < 1.2:
                last_holder = k
        if play and last_holder:
            poss_cycles[last_holder[0]] += 1
        if play:
            res['l']['ball_att_half'] += bx > 0
            res['r']['ball_att_half'] += bx < 0
            res['l']['ball_x_sum'] += bx
            res['r']['ball_x_sum'] += -bx
            res['l']['play_cycles'] += 1
            res['r']['play_cycles'] += 1
        if prev:
            _, pball, ppl = prev
            for k, p in pl.items():
                if k in ppl:
                    q = ppl[k]
                    step = math.hypot(p['x'] - q['x'], p['y'] - q['y'])
                    if step < 3:
                        dist_run[k] += step
                    if p['kick'] > q['kick'] and math.hypot(q['x'] - pball[0], q['y'] - pball[1]) < 1.25:
                        kicks.append((t, k[0], k[1], ball, mode_at[i]))
        for k, p in pl.items():
            if p['stamina'] > 0:
                min_stamina[k] = min(min_stamina[k], p['stamina'])
        prev = (t, ball, pl)

    # pases, intercepciones y tiros a partir de la secuencia de patadas
    for idx, (t, side, unum, ball, mode) in enumerate(kicks):
        r = res[side]
        r['kicks'] += 1
        bx, by, vx, vy = ball
        x, y = team_frame(side, bx, by)
        vx, vy = (vx, vy) if side == 'l' else (-vx, -vy)
        speed = math.hypot(vx, vy)
        is_shot = False
        if x > 30 and vx > 0.5 and speed > 1.6:
            ty = y + vy * (52.5 - x) / vx
            if abs(ty) < 12:
                is_shot = True
                r['shots'] += 1
                r['shots_on_target'] += abs(ty) < 7.01
        nxt = None
        for j in range(idx + 1, len(kicks)):
            if kicks[j][1:3] != (side, unum):
                nxt = kicks[j]
                break
            else:
                break                           # conducción: mismo jugador vuelve a tocar
        if is_shot or nxt is None or (nxt and nxt[0] - t > 60):
            continue
        if nxt[1] == side:
            r['passes_ok'] += 1
        else:
            r['passes_lost'] += 1

    ret = {'teams': teams, 'cycles': frames[-1][0] if frames else 0}
    total_poss = sum(poss_cycles.values()) or 1
    mode_counts = Counter(m for _, m in modes)
    for s in ('l', 'r'):
        r = res[s]
        name = teams['left'] if s == 'l' else teams['right']
        att = r['passes_ok'] + r['passes_lost']
        pc = r['play_cycles'] or 1
        runs = [v for k, v in dist_run.items() if k[0] == s]
        stams = [v for k, v in min_stamina.items() if k[0] == s and v < 9e9]
        other = 'r' if s == 'l' else 'l'
        ret[s] = dict(
            team=name,
            goals=teams['score_l'] if s == 'l' else teams['score_r'],
            possession=100.0 * poss_cycles[s] / total_poss,
            ball_in_att_half=100.0 * r['ball_att_half'] / pc,
            mean_ball_x=r['ball_x_sum'] / pc,
            kicks=int(r['kicks']),
            shots=int(r['shots']), shots_on_target=int(r['shots_on_target']),
            pass_attempts=int(att), passes_ok=int(r['passes_ok']),
            pass_accuracy=100.0 * r['passes_ok'] / att if att else 0.0,
            dist_per_player=sum(runs) / max(1, len(runs)),
            min_stamina_mean=sum(stams) / max(1, len(stams)),
            offsides=sum(c for m, c in mode_counts.items() if m == 'offside_' + s),
            fouls=sum(c for m, c in mode_counts.items()
                      if m in ('foul_charge_' + s, 'back_pass_' + s, 'free_kick_fault_' + s,
                               'catch_fault_' + s, 'foul_push_' + s)),
            corners=mode_counts.get('corner_kick_' + s, 0),
        )
    # métricas internas de los agentes (localización, tiempo de decisión)
    truth = {}
    for (t, ball, pl) in frames:
        for k, p in pl.items():
            truth[(t,) + k] = (p['x'], p['y'], p['body'])
    for s in ('l', 'r'):
        name = ret[s]['team']
        errs, aerrs, ms, decs = [], [], [], Counter()
        for fn in glob.glob(os.path.join(folder, 'agents', '%s_%s_*.json' % (name, s))):
            d = json.load(open(fn))
            for row in d['rows']:
                t, x, y, body, nf, dec, dt, st = row
                decs[dec] += 1
                ms.append(dt)
                tr = truth.get((t, s, d['unum']))
                if tr and dec not in ('KICKOFF_POS',):
                    tx, ty = team_frame(s, tr[0], tr[1])
                    tb = tr[2] if s == 'l' else (tr[2] + 180)
                    errs.append(math.hypot(x - tx, y - ty))
                    aerrs.append(abs((body - tb + 180) % 360 - 180))
        errs.sort(); aerrs.sort(); ms.sort()

        def pct(a, q):
            return a[min(len(a) - 1, int(q * len(a)))] if a else float('nan')
        ret[s].update(loc_err_mean=sum(errs) / max(1, len(errs)), loc_err_p50=pct(errs, .5),
                      loc_err_p90=pct(errs, .9), body_err_p50=pct(aerrs, .5),
                      think_ms_mean=sum(ms) / max(1, len(ms)), think_ms_p99=pct(ms, .99),
                      decisions=dict(decs.most_common()))
    return ret


if __name__ == '__main__':
    out = match_metrics(sys.argv[1])
    print(json.dumps(out, indent=1, default=float))
    if '--json' in sys.argv:
        json.dump(out, open(sys.argv[sys.argv.index('--json') + 1], 'w'), indent=1)
```